1. Definition of the Parameter Layout (APVTS)

The basic architecture of the plugin controls has been implemented, using JUCE's state management system to centralize communication between the processor and the user interface.

EQ_SharedCode → EQAudioProcessor

```cpp
juce::AudioProcessorValueTreeState::ParameterLayout EQAudioProcessor::createParameterLayout()
{
    juce::AudioProcessorValueTreeState::ParameterLayout layout;

    layout.add(std::make_unique<juce::AudioParameterFloat>("LowCut Frequency",
                                                           "Low Cut Frequency",
                                                           juce::NormalisableRange<float>(20.0f, 20000.0f, 1.0f, 1.0f),
                                                           20.0f));

    layout.add(std::make_unique<juce::AudioParameterFloat>("HighCut Frequency",
                                                           "HighCut Frequency",
                                                           juce::NormalisableRange<float>(20.0f, 20000.0f, 1.0f, 1.0f),
                                                           20000.0f));

    layout.add(std::make_unique<juce::AudioParameterFloat>("Peak Frequency",
                                                           "Peak Frequency",
                                                           juce::NormalisableRange<float>(20.0f, 20000.0f, 1.0f, 1.0f),
                                                           750.0f));

    layout.add(std::make_unique<juce::AudioParameterFloat>("Peak Gain",
                                                           "Peak Gain",
                                                           juce::NormalisableRange<float>(-24.0f, 24.0f, 0.5f, 1.0f),
                                                           0.0f));

    layout.add(std::make_unique<juce::AudioParameterFloat>("Peak Quality",
                                                           "Peak Quality",
                                                           juce::NormalisableRange<float>(0.1f, 10.0f, 0.05f, 1.0f),
                                                           1.0f));
    juce::StringArray stringArray;
    for (int i = 0; i < 4; ++i)
    {
        juce::String str;
        str << (12 + i * 12);
        str << " dB/Oct";
        stringArray.add(str);
    }

    layout.add(std::make_unique<juce::AudioParameterChoice>("LowCut Slope", "LowCut Slope", stringArray,0));
    layout.add(std::make_unique<juce::AudioParameterChoice>("HighCut Slope", "HighCut Slope", stringArray, 0));

    return layout;
}
```

Parameters and Logic:

- Frequency Bands: Three control points (LowCut, HighCut, and Peak) are defined with a range of 20 Hz to 20 kHz. A resolution of 1.0 Hz is used.

- Gain Control (Peak): Range of ±24 dB in 0.5 dB steps, allowing detailed control over the amplitude of the parametric band.

- Quality Factor (Q): Range from 0.1 to 10.0 in 0.05 steps, defining the bandwidth of the bell filter .

- Cutoff Slopes: Implementation of a discrete selection logic using a for loop that generates options of 12, 24, 36 and 48 dB/Oct for the high pass and low pass filters.

Functions and Variables Used:

- *createParameterLayout()* : Method responsible for instantiating and returning the ParameterLayout object necessary for the initialization of the AudioProcessorValueTreeState.

- *juce::AudioParameterFloat* : Class for continuous value parameters.
    - parameterID: Unique identifier string for automation.
    - parameterName: User-readable name in the DAW.
    - NormalisableRange<float>: Object that defines the behavior of the slider (min, max, interval, skew).
    - defaultValue: Initial value assigned when loading the plugin.

- *juce::AudioParameterChoice* : Class for multiple selection parameters (drop-down menus).

    - choices: StringArray that contains the text labels of the available options.
    - defaultIndex : Index of the option selected by default.

- *juce::NormalisableRange<float>* :
    - rangeStart / rangeEnd: Parameter limits.
    - intervalValue: Minimum increment of the value.
    - skewFactor: Skewness factor (1.0 for linear response)

The APVTS structure enables thread-safe parameter management, essential for synchronization between the audio and GUI threads. The use of NormalizableRange with specific steps ensures that the quantization of control values is consistent with the accuracy requirements of a professional signal processor.

2. Processing Chain (DSP Chain) Configuration

The signal processing infrastructure has been defined using JUCE's DSP module, structuring the filters into mono processing chains to manage stereo audio independently.

EQ_SharedCode | EQAudioProcessor

```
private:
    using Filter = juce::dsp::IIR::Filter<float>;

    using CutFilter = juce::dsp::ProcessorChain<Filter, Filter, Filter, Filter>;

    using MonoChain = juce::dsp::ProcessorChain<CutFilter, Filter, CutFilter>;

    MonoChain leftChain, rightChain;
```

Parameters and Logic:

- Filter Hierarchy: An alias Filter is used for single-precision IIR filters.

- CutFilter Structure: A ProcessorChain composed of 4 filters in series is defined. This architecture allows for a sum of 12 dB/Oct slopes for each filter, achieving the cuts of up to 48 dB/Oct defined in APVTS.

- Mono Chain: Final structure that organizes the signal flow in three stages: LowCut (CutFilter), Peak (Filter) and HighCut (CutFilter).

- Stereo Instantiation: Two instances of MonoChain (leftChain and rightChain) are declared to process the left and right channels separately.

Functions and Variables Used:

- *juce::dsp::IIR::Filter<float>* : Base class for infinite impulse response filters.
- *juce::dsp::ProcessorChain* : Class template that allows chaining multiple DSP processors, automatically managing the passing of samples between them.
- *using (Type Aliasing)* : Used to simplify the declaration of complex template types, improving code readability.

The implementation establishes a serial filter topology. ProcessorChain's modularity allows each band to be treated as a logical unit, facilitating the expansion of the cutoff filter slope by concatenating second-order IIR instances.

3. Initialization in prepareToPlay

The DSP engine preparation has been implemented, ensuring that all components of the audio chain know the environment specifications before playback begins.

```
EQ_SharedCode                    (Ámbito global)

//==============================================================================
void EQAudioProcessor::prepareToPlay (double sampleRate, int samplesPerBlock)
{
    // Use this method as the place to do any pre-playback
    // initialisation that you need..

    juce::dsp::ProcessSpec spec;

    spec.maximumBlockSize = samplesPerBlock;

    spec.numChannels = 1;

    spec.sampleRate = sampleRate;

    leftChain.prepare(spec);
    rightChain.prepare(spec);
}
```

Parameters and Logic:

- DSP configuration: A ProcessSpec object is used to pass the sampling rate and maximum block size to the filtering chains.

- Channel Synchronization: Each MonoChain is initialized with a single channel (numChannels = 1), as stereo processing is divided into two independent mono chains.

Functions and Variables Used:

- *prepareToPlay()* : AudioProcessor method that the DAW calls before starting audio processing.

- *juce::dsp::ProcessSpec* : Structure that contains:
    - sampleRate: Current sampling frequency (e.g. 44100 Hz).
    - maximumBlockSize: Maximum size of the audio blocks that the plugin will process.
    - numChannels : Number of channels that this specific processor will handle.

- *prepare()* : Method that allocates memory and precalculates necessary coefficients in each element of the DSP chain.

Before the music starts playing, the program needs to know the audio sample rate and how much information it will receive at once. This function gives the equalizer these instructions so it's ready and there are no errors or strange noises when you press "play."

4. Implementation of the Processing Loop (processBlock)

The core of real-time processing has been configured, where the incoming audio is broken down into mono blocks to be filtered by the corresponding DSP chains.

```
EQ_SharedCode        EQAudioProcessor        hasEditor() const
void EQAudioProcessor::processBlock (juce::AudioBuffer<float>& buffer, juce::MidiBuffer& midiMessages)
{
    juce::ScopedNoDenormals noDenormals;
    auto totalNumInputChannels  = getTotalNumInputChannels();
    auto totalNumOutputChannels = getTotalNumOutputChannels();

    // In case we have more outputs than inputs, this code clears any output
    // channels that didn't contain input data, (because these aren't
    // guaranteed to be empty - they may contain garbage).
    // This is here to avoid people getting screaming feedback
    // when they first compile a plugin, but obviously you don't need to keep
    // this code if your algorithm always overwrites all the output channels.
    for (auto i = totalNumInputChannels; i < totalNumOutputChannels; ++i)
        buffer.clear (i, 0, buffer.getNumSamples());

    juce::dsp::AudioBlock<float> block(buffer);

    auto leftBlock = block.getSingleChannelBlock(0);
    auto rightBLock = block.getSingleChannelBlock(1);

    juce::dsp::ProcessContextReplacing<float> leftContext(leftBlock);
    juce::dsp::ProcessContextReplacing<float> rightContext(rightBLock);

    leftChain.process(leftContext);
    rightChain.process(rightContext);
}
```

Parameters and Logic:

- Denormal Handling: ScopedNoDenormals is used to prevent CPU performance drops caused by denormal numbers (underflow) in recursive IIR filter calculations.

- Block Abstraction: The native AudioBuffer is converted into an AudioBlock , facilitating compatibility with the JUCE DSP module.

- Contextual Processing: Processing contexts are created for each channel, allowing filters to modify the audio directly "in-place".

Functions and Variables Used:

- *juce::ScopedNoDenormals* : Class that optimizes CPU usage during processing by avoiding the handling of numbers extremely close to zero.

- *juce::dsp::AudioBlock<float>* : Class that wraps the audio buffer data.
    - getSingleChannelBlock(index) : Extracts a pointer and metadata from a specific channel block.

- *juce::dsp::ProcessContextReplacing<float>* : Context that manages the input and output of data on the same memory address, optimizing cache access.

The processing block segments the stereo data stream into independent mono streams. The use of replacement contexts ensures maximum efficiency by processing the filter coefficients directly on the input buffer, minimizing latency and memory overhead.

5. Data Structure and Parameter Extraction

An auxiliary structure and a utility function have been defined to centralize the obtaining of values from the AudioProcessorValueTreeState, facilitating its use in the audio loop.

EQ_SharedCode | EQAudioProcessor

```cpp

struct ChainSettings
    {
    float peakFreq{ 0 }, peakGainInDecibels{ 0 }, peakQuality{ 1.0f };
    float lowCutFreq{ 0 }, highCutFreq{ 0 };
    int lowCutSlope{ 0 }, highCutSlope{ 0 };
};

ChainSettings getChainSettings(juce::AudioProcessorValueTreeState& apvts);

```

EQ_SharedCode | (Ámbito global) | getChainSettings(juce::AudioProcesso

```cpp
ChainSettings getChainSettings(juce::AudioProcessorValueTreeState& apvts)
{
    ChainSettings settings;

    settings.peakFreq = apvts.getRawParameterValue("Peak Frequency")->load();
    settings.peakGainInDecibels = apvts.getRawParameterValue("Peak Gain")->load();
    settings.peakQuality = apvts.getRawParameterValue("Peak Quality")->load();
    settings.lowCutFreq = apvts.getRawParameterValue("LowCut Frequency")->load();
    settings.highCutFreq = apvts.getRawParameterValue("HighCut Frequency")->load();
    settings.lowCutSlope = apvts.getRawParameterValue("LowCut Slope")->load();
    settings.highCutSlope = apvts.getRawParameterValue("HighCut Slope")->load();

    return settings;
}
```

Parameters and Logic:

- ChainSettings Structure : Groups all control variables (frequencies, gains, qualities and slopes) into a single object to avoid repetitive and scattered calls to the APVTS.

- The `getChainSettings` function reads the current parameter values. It uses `load()` to ensure an atomic and safe read from the audio thread.

- Type Mapping: Frequencies and gains are extracted as float, while slope indices are retrieved as int.

Functions and Variables Used:

- *apvts.getRawParameterValue()* : Gets a pointer to the real-time value of a parameter using its ID.

- *load()* : Method of std::atomic that ensures the value read is the most recent and consistent across different threads of execution.

Implementing a dedicated extraction function decouples the data retrieval logic from the processing logic. This improves maintainability and ensures that the audio thread works with a consistent "snapshot" of all controls at the start of each block.

6. Real-Time Coefficient Update

The necessary logic has been integrated to transform the control parameters into mathematical coefficients applicable to the Peak filter, both in the initialization and in the continuous process.

```
EQ_SharedCode  →  EQAudioProcessor  →  prepareToPlay(double sampleRate, int
void EQAudioProcessor::prepareToPlay (double sampleRate, int samplesPerBlock)
{
    // Use this method as the place to do any pre-playback
    // initialisation that you need..

    juce::dsp::ProcessSpec spec;

    spec.maximumBlockSize = samplesPerBlock;

    spec.numChannels = 1;

    spec.sampleRate = sampleRate;

    leftChain.prepare(spec);
    rightChain.prepare(spec);

    auto chainSettings = getChainSettings(apvts);
    auto peakCoefficients = juce::dsp::IIR::Coefficients<float>::makePeakFilter(sampleRate, chainSettings.peakFreq,
        chainSettings.peakQuality, juce::Decibels::decibelsToGain(chainSettings.peakGainInDecibels));

    *leftChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
    *rightChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
}
```

```
void EQAudioProcessor::processBlock (juce::AudioBuffer<float>& buffer, juce::MidiBuffer& midiMessages)
{
    juce::ScopedNoDenormals noDenormals;
    auto totalNumInputChannels  = getTotalNumInputChannels();
    auto totalNumOutputChannels = getTotalNumOutputChannels();

    // In case we have more outputs than inputs, this code clears any output
    // channels that didn't contain input data, (because these aren't
    // guaranteed to be empty - they may contain garbage).
    // This is here to avoid people getting screaming feedback
    // when they first compile a plugin, but obviously you don't need to keep
    // this code if your algorithm always overwrites all the output channels.
    for (auto i = totalNumInputChannels; i < totalNumOutputChannels; ++i)
        buffer.clear (i, 0, buffer.getNumSamples());

    auto chainSettings = getChainSettings(apvts);
    auto peakCoefficients = juce::dsp::IIR::Coefficients<float>::makePeakFilter(getSampleRate(), chainSettings.peakFreq,
        chainSettings.peakQuality, juce::Decibels::decibelsToGain(chainSettings.peakGainInDecibels));

    *leftChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
    *rightChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
```

Parameters and Logic:

- Coefficient Calculation: The static function makePeakFilter is used to generate the coefficients of a bell filter based on the center frequency, Q factor, and gain.

- Unit Conversion: The decibel (dB) gain is transformed to linear gain using decibelsToGain , a necessary requirement for the DSP engine.

- Chain Assignment: The specific element of the processing chain (ChainPositions::Peak) is accessed and its coefficients are updated synchronously for the left and right channels.

Functions and Variables Used:

- *juce::dsp::IIR::Coefficients<float>::makePeakFilter* : Calculates the IIR filter coefficients based on:
    - sampleRate: Sampling frequency.
    - frequency: Central frequency of the bell.
    - Q: Quality factor (bandwidth).
    - gain : Linear gain.

- *leftChain.get<Index>()* : Template function that allows access to a specific processor within a ProcessorChain at compile time.

- **coefficients = *newCoefficients* : Dereferencing operation to copy the values of the new coefficients into the pointer managed by the filter.

Updating coefficients in the processBlock allows the filter to respond instantly to changes in the interface. By replicating the same coefficients in leftChain and rightChain, identical phase and magnitude response is ensured in both channels, maintaining stereo imaging coherence.

7. Integration and Logic of the LowCut (High-Pass) Filter

LowCut filter processing has been implemented using a variable-order Butterworth filter topology. The filter slope determines how many links in the processing chain are activated to achieve the desired attenuation.

```cpp
auto chainSettings = getChainSettings(apvts);
auto peakCoefficients = juce::dsp::IIR::Coefficients<float>::makePeakFilter(sampleRate, chainSettings.peakFreq,
    chainSettings.peakQuality, juce::Decibels::decibelsToGain(chainSettings.peakGainInDecibels));

auto cutCoefficients = juce::dsp::FilterDesign<float>::designIIRHighpassHighOrderButterworthMethod(chainSettings.lowCutFreq,
                                                                                                    getSampleRate(),
                                                                                                    2 * (chainSettings.lowCutSlope + 1));

*leftChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
*rightChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;

// --- CONFIGURACIÓN LOW CUT IZQUIERDO ---
auto& leftLowCut = leftChain.get<ChainPositions::LowCut>();
leftLowCut.setBypassed<0>(true);
leftLowCut.setBypassed<1>(true);
leftLowCut.setBypassed<2>(true);
leftLowCut.setBypassed<3>(true);
```

```cpp
switch (chainSettings.lowCutSlope)
{
case Slope::Slope_12:
{
    *leftLowCut.get<0>().coefficients = *cutCoefficients[0];
    leftLowCut.setBypassed<0>(false);
    break;
}
case Slope::Slope_24:
{
    *leftLowCut.get<0>().coefficients = *cutCoefficients[0];
    leftLowCut.setBypassed<0>(false);
    *leftLowCut.get<1>().coefficients = *cutCoefficients[1];
    leftLowCut.setBypassed<1>(false);
    break;
}
case Slope::Slope_36:
{
    *leftLowCut.get<0>().coefficients = *cutCoefficients[0];
    leftLowCut.setBypassed<0>(false);
    *leftLowCut.get<1>().coefficients = *cutCoefficients[1];
    leftLowCut.setBypassed<1>(false);
    *leftLowCut.get<2>().coefficients = *cutCoefficients[2];
    leftLowCut.setBypassed<2>(false);
    break;
}
case Slope::Slope_48:
{
    *leftLowCut.get<0>().coefficients = *cutCoefficients[0];
    leftLowCut.setBypassed<0>(false);
    *leftLowCut.get<1>().coefficients = *cutCoefficients[1];
    leftLowCut.setBypassed<1>(false);
    *leftLowCut.get<2>().coefficients = *cutCoefficients[2];
    leftLowCut.setBypassed<2>(false);
    *leftLowCut.get<3>().coefficients = *cutCoefficients[3];
    leftLowCut.setBypassed<3>(false);
    break;
}
}
```

```cpp
switch (chainSettings.lowCutSlope)
{
case Slope::Slope_12:
{
    *rightLowCut.get<0>().coefficients = *cutCoefficients[0];
    rightLowCut.setBypassed<0>(false);
    break;
}
case Slope::Slope_24:
{
    *rightLowCut.get<0>().coefficients = *cutCoefficients[0];
    rightLowCut.setBypassed<0>(false);
    *rightLowCut.get<1>().coefficients = *cutCoefficients[1];
    rightLowCut.setBypassed<1>(false);
    break;
}
case Slope::Slope_36:
{
    *rightLowCut.get<0>().coefficients = *cutCoefficients[0];
    rightLowCut.setBypassed<0>(false);
    *rightLowCut.get<1>().coefficients = *cutCoefficients[1];
    rightLowCut.setBypassed<1>(false);
    *rightLowCut.get<2>().coefficients = *cutCoefficients[2];
    rightLowCut.setBypassed<2>(false);
    break;
}
case Slope::Slope_48:
{
    *rightLowCut.get<0>().coefficients = *cutCoefficients[0];
    rightLowCut.setBypassed<0>(false);
    *rightLowCut.get<1>().coefficients = *cutCoefficients[1];
    rightLowCut.setBypassed<1>(false);
    *rightLowCut.get<2>().coefficients = *cutCoefficients[2];
    rightLowCut.setBypassed<2>(false);
    *rightLowCut.get<3>().coefficients = *cutCoefficients[3];
    rightLowCut.setBypassed<3>(false);
    break;
}
}
```

Parameters and Logic:

- Butterworth Filter Design: Coefficients for a high-pass filter are generated based on the cutoff frequency (lowCutFreq) extracted from the APVTS.

- Filter Order Calculation: The function requires the mathematical filter order. Since each option in our menu (Slope_12, Slope_24, etc.) represents an index (0, 1, 2, 3), the formula 2 * (chainSettings.lowCutSlope + 1) translates this index into the correct orders (2, 4, 6, 8 respectively).

- Chain Management (Bypass): Our CutFilter consists of up to 4 individual filters (each contributing 12 dB/Oct). By default, all filters are bypassed (off) at the beginning of the block.

Functions and Variables Used:

- *juce::dsp::FilterDesign<float>::designIIRHighpassHighOrderButterworthMethod* : Returns a ReferenceCountedArray containing the coefficients for the various cascaded IIR filters needed to form the higher-order filter.

- *setBypassed<Index>(bool)* : Method of juce::dsp::ProcessorChain that activates (false) or deactivates (true) a specific processor within the chain without taking it out of the memory flow, saving CPU when steep slopes are not needed.

- Coefficient Dereferencing (**cutCoefficients[i])*: Extracts the specific coefficient block from the generated array to assign it to position i of our LowCut substring.

Variable slope implementation is achieved by cascading multiple second-order (biquadratic) IIR filters. By using the Butterworth approximation, we guarantee the flattest possible frequency response in the passband (no ripple) at the expense of a nonlinear phase transition. Dynamically disabling unused nodes with `setBypassed` optimizes CPU cycles, ensuring that the audio engine only processes the mathematical operations strictly necessary for the selected slope (e.g., 1 filter for 12 dB/Oct, 4 filters for 48 dB/Oct).